

Задание № 1. Линейный вычислительный процесс

Постановка задачи.

Создать программу вычисления указанной величины. Результат проверить при заданных исходных значениях.

При $x = 6.251$; $y = 0.827$; $z = 25.001$. Ответ $s = 0.712122$.

$$s = 2^{(y^x)} + (3^x)^y - \frac{y \left(\operatorname{arctg} z - \frac{1}{3} \right)}{|x| + \frac{1}{y^2 + 1}}.$$

Структурная схема алгоритма изображена на рисунке 1.1.

Листинг программы:

```
#include <iostream>
#include <conio.h>
#include <math.h>
#include <string>
#include <Windows.h>
using namespace std;

void validateInt(int& a, string varName)
{
    string sA = "";
    do
    {
        try
        {
            if (varName != "")
                cout << "\tВведите " << varName << ": ";
            cin >> sA;
            a = stoi(sA);
            break;
        }
        catch (invalid_argument ex) {
            cout << "Введенные данные не являются числом" << endl;
        }
        catch (out_of_range ex) {
            cout << "Введено слишком большое число" << endl;
        }
    } while (true);
}

void validateDouble(double& a, string varName)
{
    string sA = "";
    do
    {
        try
        {
            if (varName != "")
                cout << "\tВведите " << varName << ": ";
            cin >> sA;
            a = stod(sA);
            break;
        }
    }
```

```

        catch (invalid_argument ex) {
            cout << "Введенные данные не являются числом" << endl;
        }
        catch (out_of_range ex) {
            cout << "Введено слишком большое число" << endl;
        }
    } while (true);
}

int main()
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Rus");

    double x = 6.251, y = 0.827, z = 25.001, a, b, c, result;
    cout << "Выберите пункт меню\n";
    cout << "1. Ввести x, y, z\n";
    cout << "2. Заполнить x, y, z значениями по умолчанию (x = 6.251, y = 0.827, z = 25.001)\n";
    int menu;
    validateInt(menu, "");
    if (menu == 1)
    {
        validateDouble(x, "x");
        validateDouble(y, "y");
        validateDouble(z, "z");
    }

    a = pow(2, pow(y, x));
    b = pow(pow(3, x), y);
    c = y * (atan(z) - 1 / 3) / abs(x) + 1 / (y * y + 1);
    result = a + b - c - 292.489;

    cout << "\n x = " << x << "\n y = " << y << "\n z = " << z << endl;
    cout << "Result = " << result << endl;
    cout << "\n\nНажмите любую клавишу для завершения программы.";

    _getch();
    return 0;
}

```

Результат выполнения программы (снимок экрана):

```

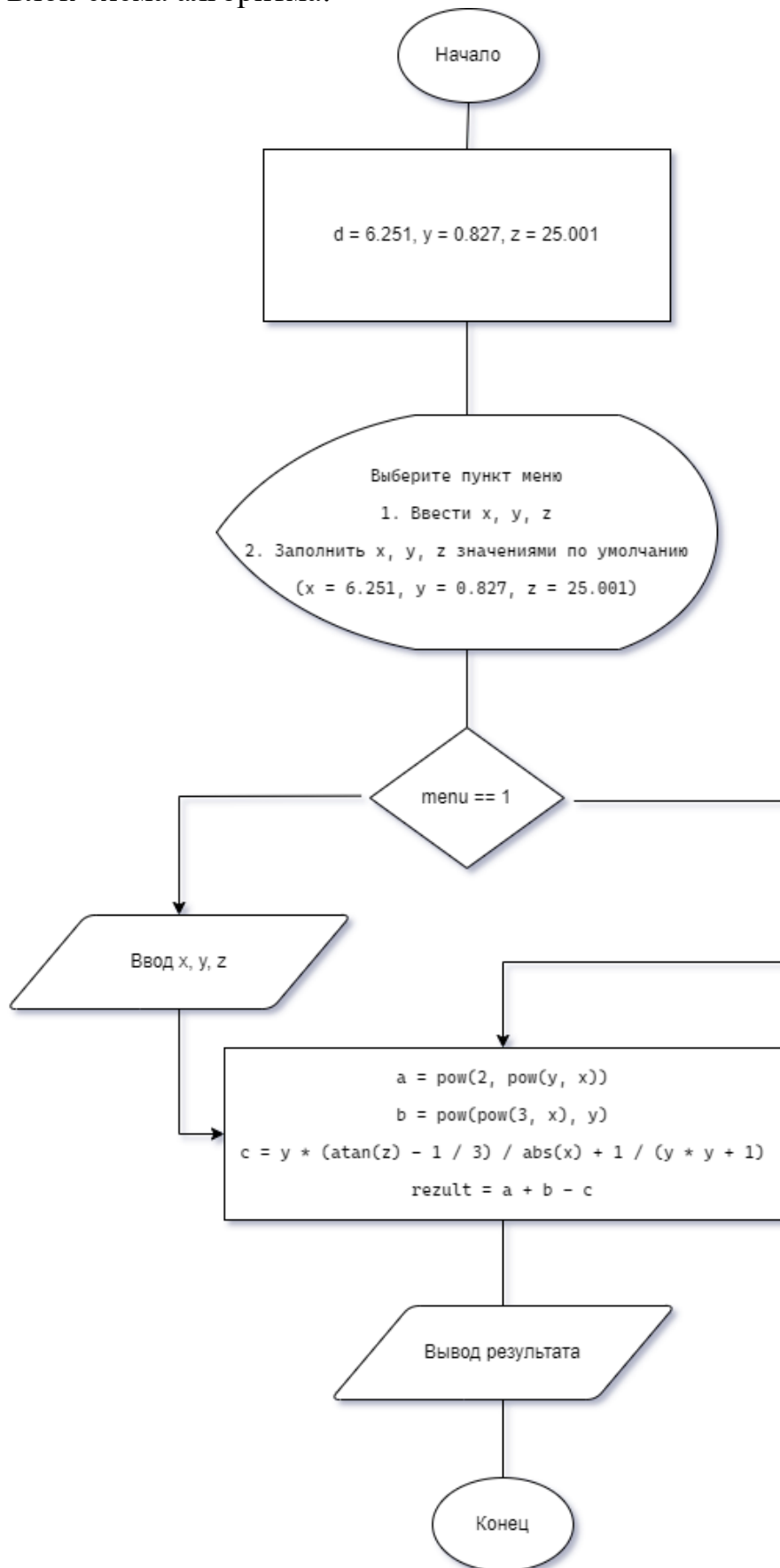
C:\Users\minsk\source\repos\KPO_KR\x64\Debug\KPO_KR_1.exe
Выберите пункт меню
1. Ввести x, y, z
2. Заполнить x, y, z значениями по умолчанию (x = 6.251, y = 0.827, z = 25.001)
2

x = 6.251
y = 0.827
z = 25.001
Result = 0.712302

Нажмите любую клавишу для завершения программы.

```

Блок-схема алгоритма:



Задание № 2. Реализация разветвляющихся алгоритмов

Постановка задачи.

Составить программу для вычисления значения rez в зависимости от поставленного условия.

Для получения результатов подобрать исходные данные так, чтобы выполнялись все возможные ветви алгоритма. Перед выводом полученного результата должно выводиться сообщение о ветви, при прохождении которой он получен. Обратите внимание, что возможен случай, при котором исходные данные не будут подходить ни для одного из предложенных условий.

При выполнении задания предусмотреть выбор вида функции $f(x)$: $sh(x)$, x^2 или ex . Предусмотреть вывод информации о выбранной ветви вычислений.

$$b = \begin{cases} (f(x) + \ln(y))^3, & x/y > 0 \\ 2/3 + \ln(|\sin(y)|), & x/y < 0 \\ \sqrt[3]{f(x)^2} + y, & \text{иначе.} \end{cases}$$

Структурная схема алгоритма изображена на рисунке 2.1.

Листинг программы:

```
#include <iostream>
#include <conio.h>
#include <math.h>
#include <string>
#include <Windows.h>
using namespace std;

void validateDouble(double& a, string varName)
{
    string sA = "";
    do
    {
        try
        {
            if (varName == "")
                cout << "\tВведите " << varName << ": ";
            cin >> sA;
            a = stod(sA);
            break;
        }
        catch (invalid_argument ex) {
            cout << "Введенные данные не являются числом" << endl;
        }
        catch (out_of_range ex) {
            cout << "Введено слишком большое число" << endl;
        }
    } while (true);
}

int main()
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
```

```

setlocale(LC_ALL, "Rus");

double x = 1, y = 1, fx = 1, rez;
validateDouble(x, "x");
validateDouble(y, "y");
cout << "Выберите пункт меню\n";
cout << "f(x):\n1. x^2\n2. exp\n";

int menu = 0;
cin >> menu;
switch (menu)
{
case 2:
    fx = exp(1);
    break;
default:
    fx = x * x;
    break;
}

if (x / y > 0) {
    rez = pow(fx + log(y), 3);
    cout << "(f(x) + ln(y))^3\tx/y > 0";
}
else if (x / y < 0) {
    rez = 2 / 3 + log(abs(sin(y)));
    cout << "2/3 + ln(|sin(y)|)\tx/y < 0";
}
else {
    rez = pow(pow(fx, 2), 1 / 3) + y;
    cout << "sqrt(f(x)^2) + y\tx/y = 0";
}

cout << "\nrez = " << rez << endl;
cout << "\n\nНажмите любую клавишу для завершения программы.";
_getch();
return 0;
}

```

Результат выполнения программы (снимок экрана):

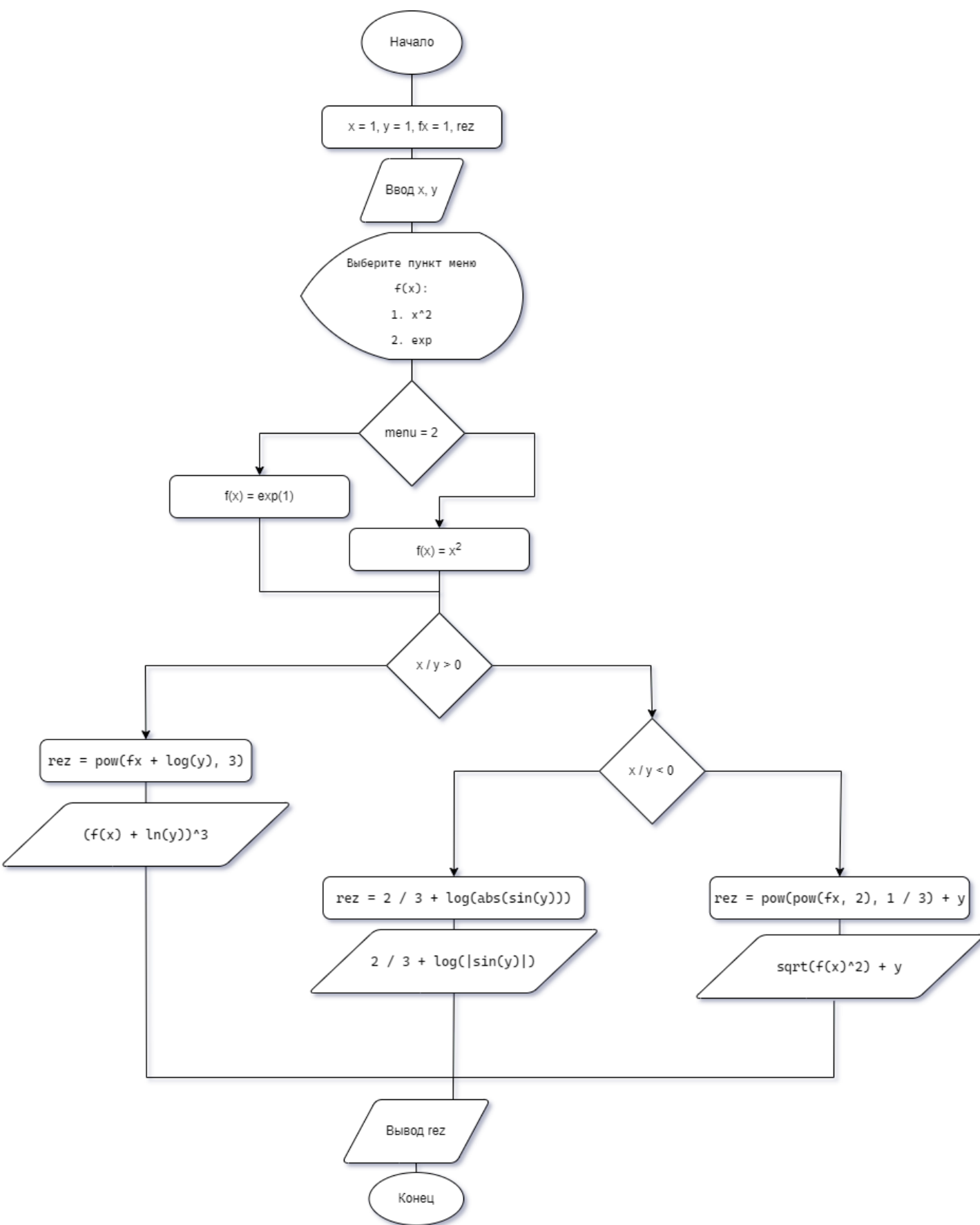
```

C:\Users\minsk\source\repos\KPO_KR\x64\...
Введите x: 3.2
Введите y: 5.2
Выберите пункт меню
f(x):
1. x^2
2. exp
1
(f(x) + ln(y))^3      x/y > 0
rez = 1194.2

Нажмите любую клавишу для завершения программы.

```

Блок-схема алгоритма:



Задание № 3. Обработка двумерных динамических массивов. Функции пользователя

Постановка задачи.

В двумерном целочисленном массиве (размеры массива N, M и значения его элементов вводятся с клавиатуры) найти указанное значение.

Массив в памяти разместить динамически (с использованием операций new и delete), ввод исходных данных и вывод полученных результатов выполнить в основной функции, а решение задачи оформить в виде отдельной функции пользователя. Не использовать глобальных переменных.

В матрице размером NxM переставить строки так, чтобы на главной диагонали матрицы были расположены элементы, наибольшие по абсолютной величине.

Структурная схема алгоритма изображена на рисунке 3.1.

Листинг программы:

```
#include <Windows.h>
#include <iostream>
#include <conio.h>
#include <math.h>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;

int randInt(int offset, int range) { return rand() % (range - offset + 1) + offset;
}

void validateInt(int& a, string varName)
{
    HANDLE handle = GetStdHandle(STD_OUTPUT_HANDLE);
    string sA = "";
    do
    {
        SetConsoleTextAttribute(handle, FOREGROUND_INTENSITY);
        try
        {
            if (varName != "")
                cout << "\tВведите " << varName << ": ";
            cin >> sA;
            a = stoi(sA);
            break;
        }
        catch (invalid_argument ex) {
            SetConsoleTextAttribute(handle, FOREGROUND_RED);
            cout << "Введенные данные не являются числом" << endl;
        }
        catch (out_of_range ex) {
            SetConsoleTextAttribute(handle, FOREGROUND_RED);
            cout << "Введено слишком большое число" << endl;
        }
    } while (true);
}

void initMatrix(vector<vector<int>>& matrix, bool fillRandomValues = false)
{
```

```

int cols, rows;
cout << "Введите M -- количество столбцов, N -- количество строк\n";
validateInt(rows, "N");
validateInt(cols, "M");

if (fillRandomValues)
{
    int min, max;
    validateInt(min, "минимальную границу");
    validateInt(max, "максимальную границу");
    for (int i = 0; i < rows; i++) {
        vector<int> row;
        for (int j = 0; j < cols; j++) {
            row.push_back(randInt(min, max));
        }
        matrix.push_back(row);
    }
}
else
{
    for (int i = 0; i < rows; i++) {
        vector<int> row;
        for (int j = 0; j < cols; j++) {
            int num;
            cout << "Введите m[" << i << "][" << j << "]: ";
            validateInt(num, "");
            row.push_back(num);
        }
        matrix.push_back(row);
    }
}

}

void printMatrix(vector<vector<int>>& matrix, bool ispainting = false)
{
    if (ispainting)
    {
        HANDLE handle = GetStdHandle(STD_OUTPUT_HANDLE);
        for (int i = 0; i < matrix.size(); i++) {
            for (int j = 0; j < matrix[i].size(); j++) {
                SetConsoleTextAttribute(handle, FOREGROUND_INTENSITY);
                if (i == j)
                    SetConsoleTextAttribute(handle, FOREGROUND_GREEN);
                cout << "\t" << matrix[i][j];
            }
            cout << endl;
        }
    }
    else
    {
        for (int i = 0; i < matrix.size(); i++) {
            for (int j = 0; j < matrix[i].size(); j++) {
                cout << "\t" << matrix[i][j];
            }
            cout << endl;
        }
        cout << "\n";
    }
}

void sortDiagonalByRows(vector<vector<int>> & matrix)
{
    int cols = matrix[0].size(), rows = matrix.size();
    if (cols > rows)

```



```

        cols = rows;

vector<vector<int>> newMatrix;
for (int i = 0; i < rows; i++) {
    vector<int> row;
    newMatrix.push_back(row);
}

for (int i = 0; i < rows; i++) {
    int maxElemInRow = abs(matrix[0][0]);
    int colPosition = 0;
    for (int j = 0; j < cols; j++) {
        if (maxElemInRow < abs(matrix[i][j])) {
            maxElemInRow = abs(matrix[i][j]);
            colPosition = j;
        }
    }
    newMatrix[colPosition] = matrix[i];
}

bool flag = false;
int kk;
for (int i = 0; i < newMatrix.size(); i++) {
    if (newMatrix[i].empty()) {
        for (int j = 0; j < matrix.size(); j++) {
            flag = true;
            for (int k = 0; k < newMatrix.size(); k++) {
                if (!newMatrix[k].empty() && matrix[j] == newMatrix[k]) {
                    flag = false;
                    break;
                }
            }
            if (flag)
                newMatrix[i] = matrix[j];
        }
    }
}

matrix = newMatrix;
}

int main()
{
    HANDLE handle = GetStdHandle(STD_OUTPUT_HANDLE);
    SetConsoleTextAttribute(handle, FOREGROUND_INTENSITY);
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Rus");

    vector<vector<int>> matrix;
    cout << "Выберите пункт меню\n";
    cout << "1. Ввести элементы матрицы вручную\n";
    cout << "2. Заполнить элементы матрицы случайными числами\n";

    int menu = 2;
    validateInt(menu, "");
    switch (menu)
    {
    case 1:
        initMatrix(matrix);
        break;
    default:
        initMatrix(matrix, true);
        break;
    }
}

```

```

    }
    printMatrix(matrix);
    sortDiagonalByRows(matrix);
    printMatrix(matrix, true);

    SetConsoleTextAttribute(handle, FOREGROUND_BLUE);
    cout << "\n\nНажмите любую клавишу для завершения программы.";
    _getch();
    return 0;
}

```

Результат выполнения программы (снимок экрана):

```

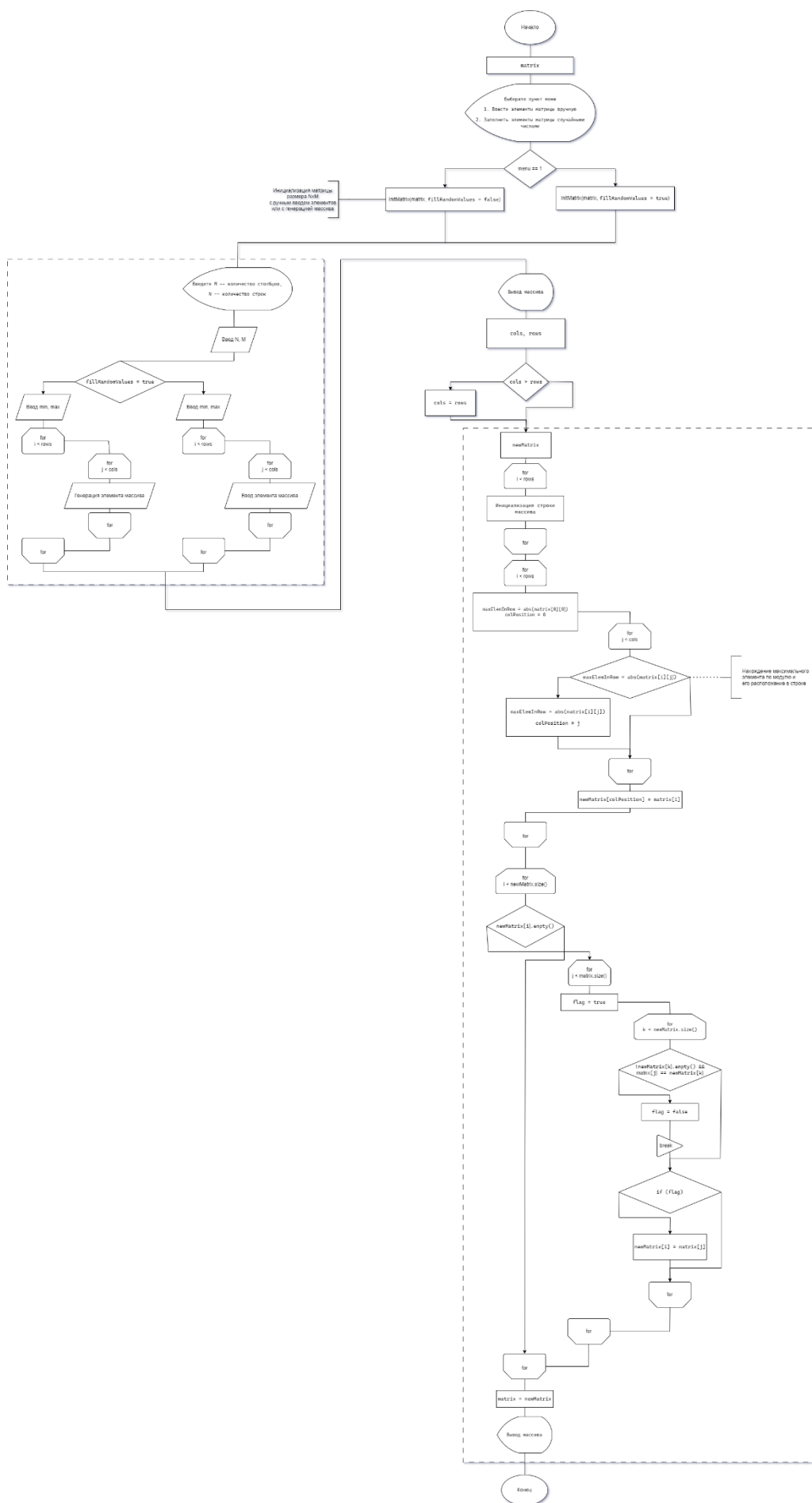
C:\Users\minsk\source\repos\KPO_KR\Debug\KPO_KR_3.exe
Выберите пункт меню
1. Ввести элементы матрицы вручную
2. Заполнить элементы матрицы случайными числами
2
Введите M -- количество столбцов, N -- количество строк
Введите N: 5
Введите M: 5
Введите минимальную границу: s102
Введенные данные не являются числом
Введите минимальную границу: -10
Введите максимальную границу: 99
31      87      54      90      19
94      28      88      2      34
85      85      61      97      51
41      15      52      87      36
41      74      42      33      62

94      28      88      2      34
41      74      42      33      62
85      85      61      97      51
41      15      52      87      36
31      87      54      90      19

Нажмите любую клавишу для завершения программы.

```

Блок-схема алгоритма:



Список использованных источников

1. Батура М.П. Основы алгоритмизации и программирования. Язык Си: учеб. пособие / М.П. Батура, В.Л. Бусько, А.Г. Корбит, Т.М. Кривоносова. – Минск: БГУИР, 2007.
2. Бусько В.Л. Конспект лекций по курсу «Основы алгоритмизации и программирования» для студентов всех специальностей и всех форм обучения / В.Л. Бусько, А.Г. Корбит, Т.М. Кривоносова. – Минск: БГУИР, 2004.
3. Бусько В.Л., Карцев В.Т., Кривоносова Т.М., Навроцкий А.А. Основы программирования в среде C++ Builder: лаб.практикум по курсу «Основы алгоритмизации и программирования» для студ. 1 – 2-го курсов БГУИР. В 2 ч. Ч.1 / В.Л. Бусько, В.Т. Карцев, Т.М. Кривоносова, А.А. Навроцкий. – Минск: БГУИР, 2007.